

Course: Cloud and Network Security CNS-C2-2026

Student Name: James Kirika

Student No: CS-CNS12-26028

September 21, 2026

Week 2 Assignment 1:

Network Traffic Analysis

Objectives.....	4
Introduction	4
Part 1: Test basic tcpdump commands/swirches.....	4
To validate if the tcpdump exists on our host.....	4
Listing Available Interfaces and Choosing an Interface to Capture From	5
Disable Name Resolution	5
Display the Ethernet Header	6
Include ASCII and Hex Output.....	6
Tcpdump Switch Combinations.....	7
Save our PCAP Output to a File	7
Reading Output from a File	8
Save a Capture to a .PCAP file	8
Read the Capture from a .PCAP file.	9
Filtering and Advanced Syntax Options	9
Host Filter	9
Utilizing Source With Port as a Filter	10
Tasks	10
Read a capture from a file without filters implemented.	10
Filter out traffic.	11
Summary	11
Part 2: Tshark commands/swirches(Analysis with Wireshark)	12
Locating Wireshark	12
Locating TShark and Selecting an Interface & Writing to a File	12
Applying Filters.....	13
Performing our first capture in Wireshark	14
Filter List.....	14
Summary	14
Part 3: Test and apply Gui wireshark filters.	15
Apply http filter	15
TCP Stream.....	15
Packet Inception, Dissecting Network Traffic With Wireshark	15
Utilizing Wireshark-lab-2.zip	15

Filter the results.	16
Follow the stream and extract the item(s) found.	16
apply http && image-jfif as a display filter.	17
Check for the presence of JFIF files in the HTTP traffic.	18
Live Capture and Analysis Connectivity to Lab	18
Check XfreeRDP.	18
Running vpn	19
Connectivity to Lab	19
Running wireshark in rdp	20
Self Analysis.....	20
ftp filter	20
Ftp-Data filter	21
ftp-data Tcp Stream.....	21
saving raw data	22
HTTP Analysis	22
Apply the filter “http && image-jfif”	22
Investigate which employee is the suspect.....	23
Further investigation	24
View conversations	24
View protocol hierarchy	24
Summary	25
Conclusion.....	25
Links:	25

Objectives

1. Part 1: Test basic tcpdump commands/swirches
2. Part 2: Test basic wireshark/tshark commands/swirches
3. Part 3: Test and apply Gui wireshark filters.

Introduction

This report documents my work through the "Intro to Network Traffic Analysis" module on HTB Academy, covering tcpdump fundamentals, the Wireshark GUI, and hands-on packet analysis across both static pcap files and a live network capture. The exercises progressed from basic interface selection and filtering to more applied scenarios: extracting a file transferred over FTP, identifying credentials exposed in cleartext HTTP traffic, and correlating suspicious activity back to a specific host. This report captures the steps taken, the commands and filters used, and the reasoning behind each finding.

Part 1: Test basic tcpdump commands/swirches

To validate if the tcpdump exists on our host

The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the following commands and output:

```
kali@kali: ~  
$ which tcpdump  
/usr/bin/tcpdump  
  
$ --version  
--version: command not found  
  
$ sudo tcpdump --version  
Command 'sude' not found, did you mean:  
command 'sure' from deb python3-sure  
command 'sudo' from deb sudo  
Try: sudo apt install <deb name>  
  
$ sudo tcpdump --version  
[sudo] password for kali:  
tcpdump version 4.99.6  
libpcap version 1.10.6 (64-bit time_t, with TPACKET_V3)  
OpenSSL 3.6.2 7 Apr 2026  
64-bit build, 64-bit time_t
```

On the right, a web browser window displays a page titled "Locate Tcpdump" with the following text:

It should be noted that Windows had a port of TCPDump called Windump. Support for windump has ceased. As an alternative running a Linux distribution such as Parrot or Ubuntu in Windows Subsystem for Linux can be an easy way to have a Linux virtual host right on our computer, allowing for the use of TCPDump and many other Linux built tools.

Locate Tcpdump

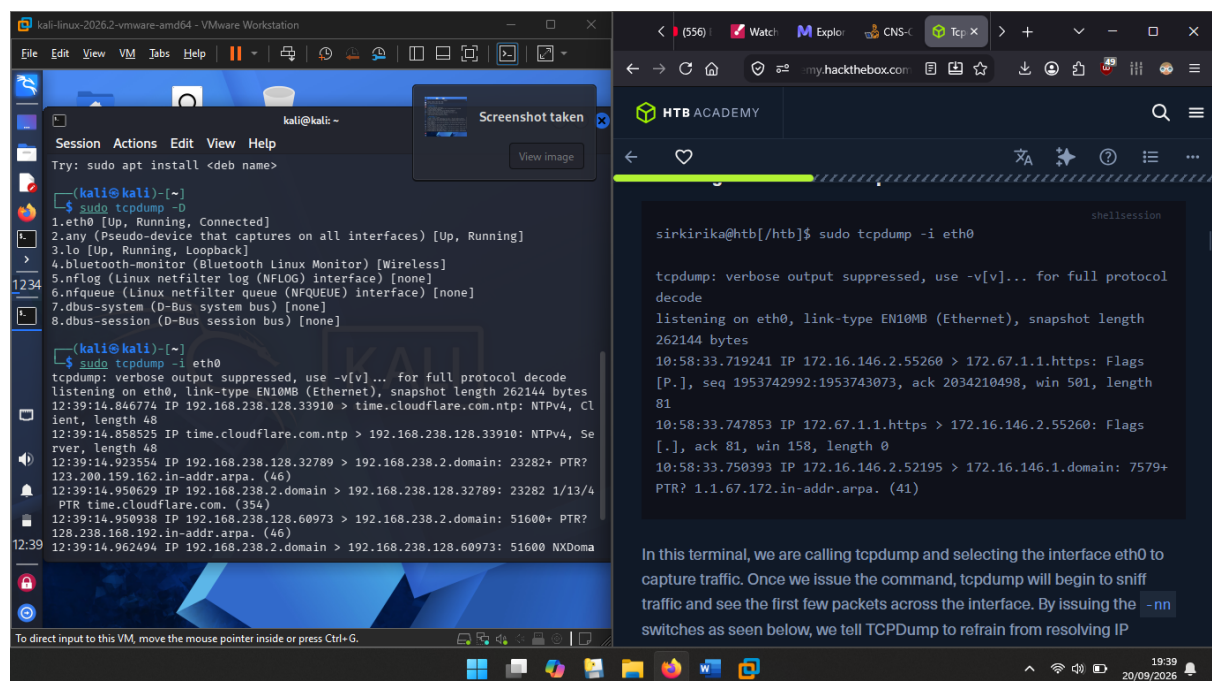
To validate if the package exists on our host, use the following command:

```
sirkirika@htb[/htb]$ which tcpdump
```

Often it can be found in `/usr/sbin/tcpdump`. However, if the package does not exist, we can install it with:

Install Tcpdump

Listing Available Interfaces and Choosing an Interface to Capture From



The screenshot shows a Kali Linux terminal window on the left and a web browser on the right. The terminal displays the command `sudo tcpdump -D` which lists available interfaces: `1.eth0 [Up, Running, Connected]`, `2.any [Pseudo-device that captures on all interfaces] [Up, Running]`, `3.lo [Up, Running, Loopback]`, `4.bluetooth-monitor [Bluetooth Linux Monitor] [Wireless]`, `5.nflog [Linux netfilter log (NFLOG) interface] [none]`, `6.nfqueue [Linux netfilter queue (NFQUEUE) interface] [none]`, `7.dbus-system [D-Bus system bus] [none]`, and `8.dbus-session [D-Bus session bus] [none]`. Below this, the command `sudo tcpdump -i eth0` is shown, followed by a list of captured packets. The web browser on the right shows the HTB Academy website with a terminal window displaying the command `sirkirika@htb[/htb]$ sudo tcpdump -i eth0` and its output, which is identical to the terminal output on the left. A text box on the right explains that the command captures traffic on `eth0` and that the `-nn` switch is used to prevent IP resolution.

Session Actions Edit View Help

Try: `sudo apt install <deb name>`

(kali@kali)-[~]

`$ sudo tcpdump -D`

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

12:39:14.846774 IP 192.168.238.128.33910 > time.cloudflare.com.ntp: NTPv4, Client, length 48

12:39:14.858525 IP time.cloudflare.com.ntp > 192.168.238.128.33910: NTPv4, Server, length 48

12:39:14.923554 IP 192.168.238.128.32789 > 192.168.238.2.domain: 23282+ PTR? 123.200.159.162.in-addr.arpa. (46)

12:39:14.950629 IP 192.168.238.2.domain > 192.168.238.128.32789: 23282 1/13/4 PTR time.cloudflare.com. (354)

12:39:14.950938 IP 192.168.238.128.60973 > 192.168.238.2.domain: 51600+ PTR? 128.238.168.192.in-addr.arpa. (46)

12:39:14.962494 IP 192.168.238.2.domain > 192.168.238.128.60973: 51600 NXDomain, length 48

12:39

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

HTB ACADEMY

sirkirika@htb[/htb]\$ sudo tcpdump -i eth0

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

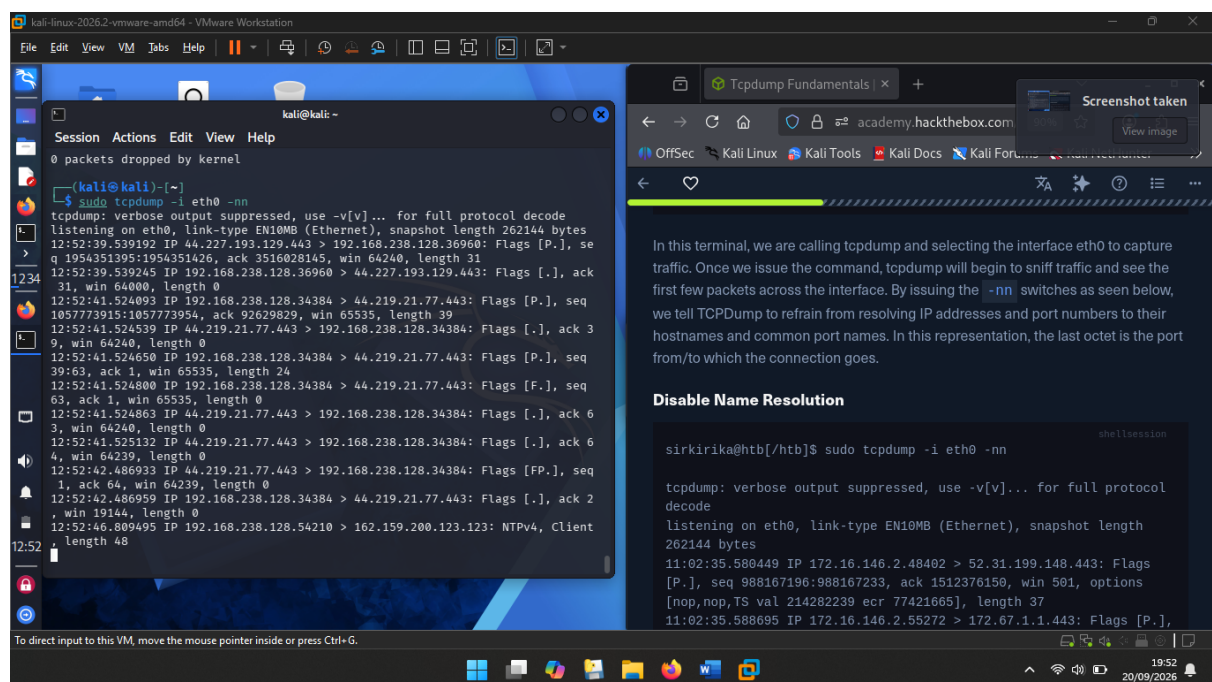
10:58:33.719241 IP 172.16.146.2.55260 > 172.67.1.1.https: Flags [P.], seq 1953742992:1953743073, ack 2034210498, win 501, length 81

10:58:33.747853 IP 172.67.1.1.https > 172.16.146.2.55260: Flags [.], ack 81, win 158, length 0

10:58:33.750393 IP 172.16.146.2.52195 > 172.16.146.1.domain: 7579+ PTR? 1.1.67.172.in-addr.arpa. (41)

In this terminal, we are calling tcpdump and selecting the interface eth0 to capture traffic. Once we issue the command, tcpdump will begin to sniff traffic and see the first few packets across the interface. By issuing the `-nn` switches as seen below, we tell TCPDump to refrain from resolving IP

Disable Name Resolution



The screenshot shows a Kali Linux terminal window on the left and a web browser on the right. The terminal displays the command `sudo tcpdump -i eth0 -nn` which captures traffic without resolving names. The output shows IP addresses instead of hostnames. The web browser on the right shows the academy.hackthebox.com website with a terminal window displaying the command `sirkirika@htb[/htb]$ sudo tcpdump -i eth0 -nn` and its output, which is identical to the terminal output on the left. A text box on the right explains that the `-nn` switch is used to prevent IP resolution and port numbers to their hostnames and common port names. In this representation, the last octet is the port from/to which the connection goes.

Session Actions Edit View Help

0 packets dropped by kernel

(kali@kali)-[~]

`$ sudo tcpdump -i eth0 -nn`

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

12:52:39.539192 IP 44.227.193.129.443 > 192.168.238.128.36960: Flags [P.], seq 1954351395:1954351426, ack 3516028145, win 64240, length 31

12:52:39.539245 IP 192.168.238.128.36960 > 44.227.193.129.443: Flags [.], ack 31, win 64000, length 0

12:52:41.524093 IP 192.168.238.128.34384 > 44.219.21.77.443: Flags [P.], seq 1057773915:1057773954, ack 92629829, win 65535, length 39

12:52:41.524539 IP 44.219.21.77.443 > 192.168.238.128.34384: Flags [.], ack 39, win 64240, length 0

12:52:41.524650 IP 192.168.238.128.34384 > 44.219.21.77.443: Flags [P.], seq 39:63, ack 1, win 65535, length 24

12:52:41.524800 IP 192.168.238.128.34384 > 44.219.21.77.443: Flags [F.], seq 63, ack 1, win 65535, length 0

12:52:41.524863 IP 44.219.21.77.443 > 192.168.238.128.34384: Flags [.], ack 63, win 64240, length 0

12:52:41.525132 IP 44.219.21.77.443 > 192.168.238.128.34384: Flags [.], ack 64, win 64239, length 0

12:52:42.486933 IP 44.219.21.77.443 > 192.168.238.128.34384: Flags [FP.], seq 1, ack 64, win 64239, length 0

12:52:42.486959 IP 192.168.238.128.34384 > 44.219.21.77.443: Flags [.], ack 2, win 19144, length 0

12:52:46.809495 IP 192.168.238.128.54210 > 162.159.200.123.123: NTPv4, Client, length 48

12:52

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Tcpdump Fundamentals

academy.hackthebox.com

Screenshot taken

View image

In this terminal, we are calling tcpdump and selecting the interface eth0 to capture traffic. Once we issue the command, tcpdump will begin to sniff traffic and see the first few packets across the interface. By issuing the `-nn` switches as seen below, we tell TCPDump to refrain from resolving IP addresses and port numbers to their hostnames and common port names. In this representation, the last octet is the port from/to which the connection goes.

Disable Name Resolution

sirkirika@htb[/htb]\$ sudo tcpdump -i eth0 -nn

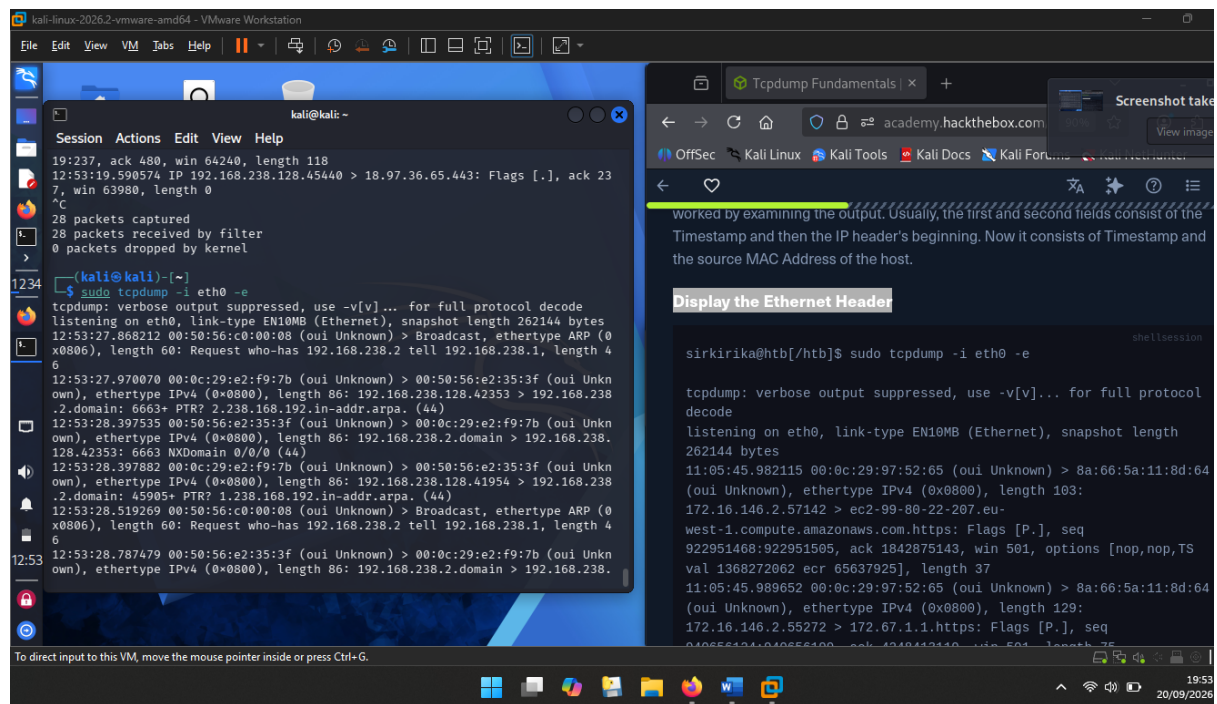
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

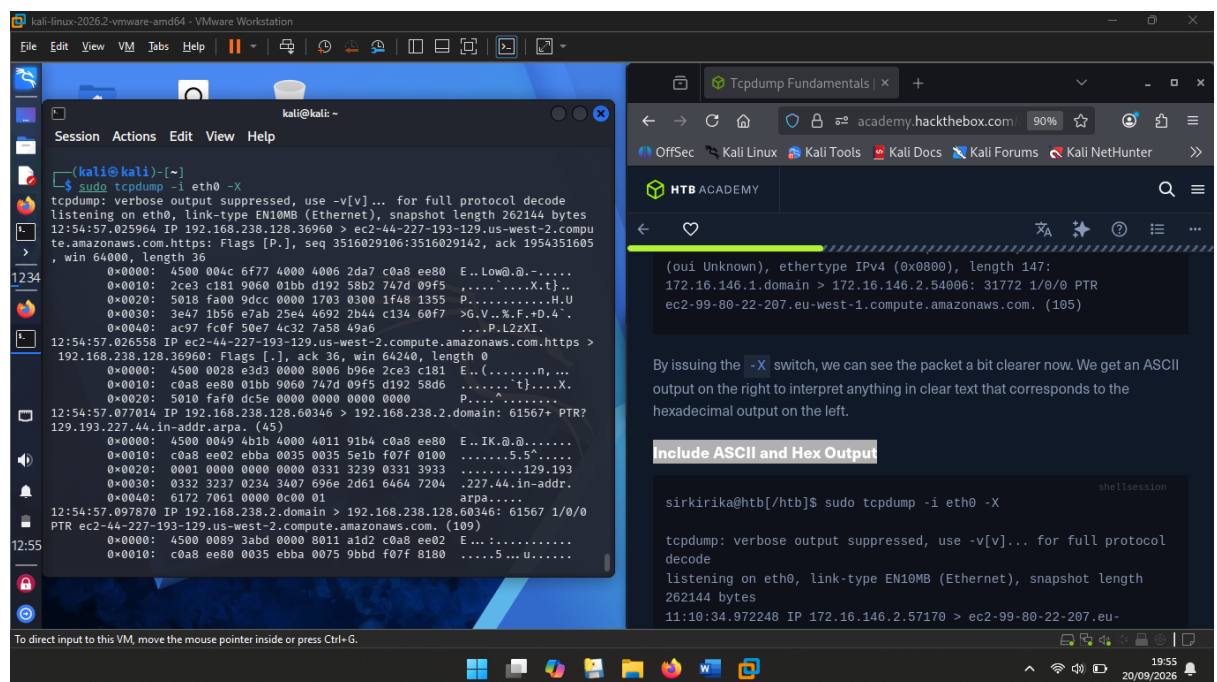
11:02:35.580449 IP 172.16.146.2.48402 > 52.31.199.148.443: Flags [P.], seq 988167196:988167233, ack 1512376150, win 501, options [nop,nop,TS val 214282239 ecr 77421665], length 37

11:02:35.588695 IP 172.16.146.2.55272 > 172.67.1.1.443: Flags [P.],

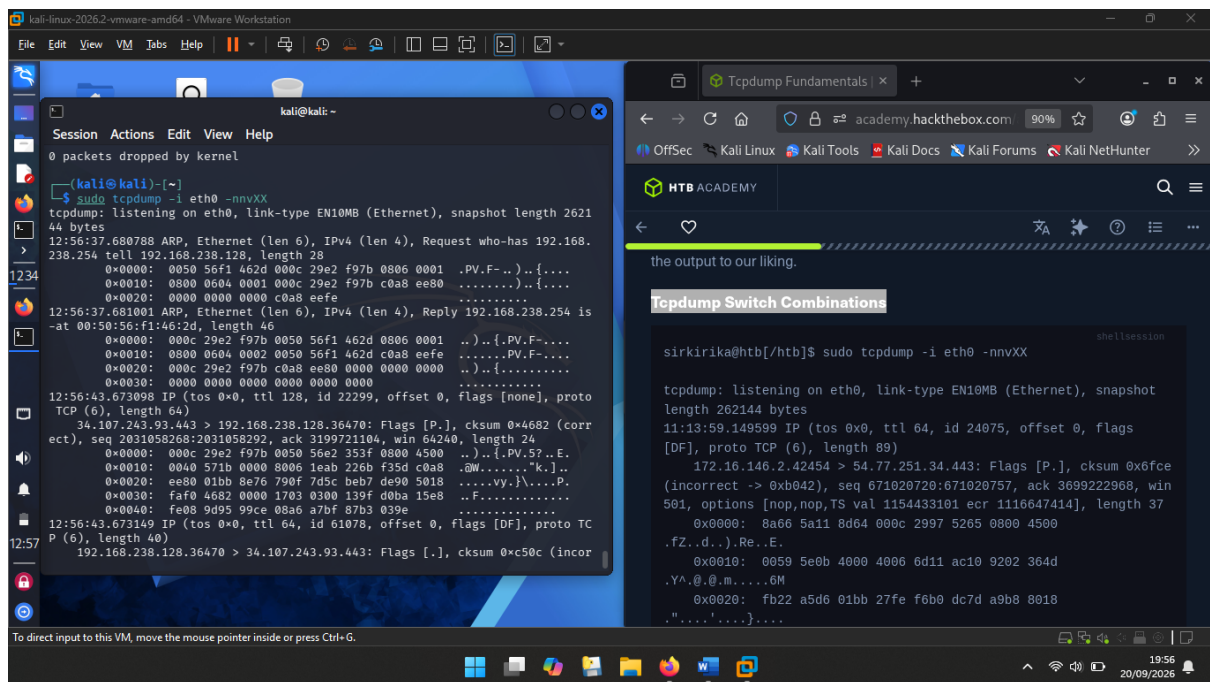
Display the Ethernet Header



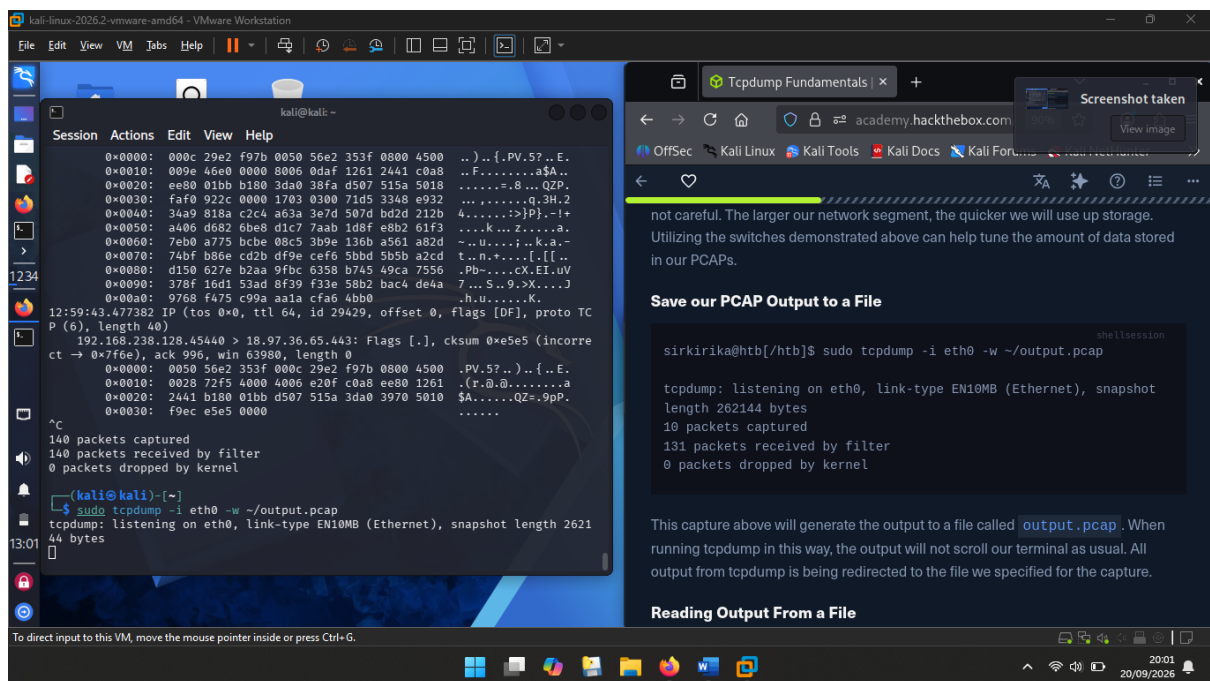
Include ASCII and Hex Output



Tcpdump Switch Combinations



Save our PCAP Output to a File



Reading Output from a File

The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the output of the command `sudo tcpdump -i eth0 -r ~/output.pcap`. The output shows several network packets, including HTTP and HTTPS traffic. On the right, a web browser window is open to the article "Reading Output From a File" on the website `academy.hackthebox.com`. The article text explains that when running tcpdump in this way, the output is redirected to the file specified for the capture.

Reading Output From a File

```
sirkirika@htb[/htb]$ sudo tcpdump -r ~/output.pcap
```

```
reading from file /home/trey/output.pcap, link-type EN10MB (Ethernet), snapshot length 262144
11:15:40.321509 IP 172.16.146.2.57236 > ec2-99-80-22-207.eu-west-1.compute.amazonaws.com.https: Flags [P.], seq 2751910362:2751910399, ack 946558143, win 501, options [nop,nop,TS val 1368866401 ecr 65790024], length 37
11:15:40.337302 IP 172.16.146.2.55416 > 172.67.1.1.https: Flags [P.], seq 3766493458:3766493533, ack 4098207917, win 501, length 75
11:15:40.398103 IP 172.67.1.1.https > 172.16.146.2.55416: Flags [.], ack 75, win 73, length 0
11:15:40.457416 IP ec2-99-80-22-207.eu-west-1.compute.amazonaws.com.https > 172.16.146.2.57236: Flags [.], ack 37, win 118, options [nop,nop,TS val 65799068 ecr 1368866401], length 0
11:15:40.458582 IP ec2-99-80-22-207.eu-
```

Save a Capture to a .PCAP file

The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the output of the command `sudo tcpdump -i eth0 -nw ~/demo.pcap`. The output shows network traffic being captured in real-time. On the right, a web browser window is open to the "Task #5" section of the article "Reading Output From a File" on the website `academy.hackthebox.com`. The task instructions ask the user to read the PCAP file into tcpdump and modify the view of the PCAP to help determine what is happening.

Task #5

Read the Capture from a .PCAP file.

Our team members have given us a PCAP they captured while surveying another section of the enterprise, read the PCAP file into tcpdump, and modify our view of the PCAP to help us determine what is happening. We can disable hostname and port resolution for simplicity and ensure we see any TCP sequence and acknowledgment numbers in absolute values. For the sake of the lab, utilize the PCAP file we created in the previous step for this task.

► Click to show answer

When done with the tasks above, please answer the questions at the bottom of the section to test our understanding.

Read the Capture from a .PCAP file.

The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the following commands and output:

```
kali@kali:~$ sudo tcpdump -i eth0 -nnw ~/dem2o.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
76 packets captured
0 packets dropped by kernel

kali@kali:~$ sudo tcpdump -nnSxR ~/dem2o.pcap
reading from file /home/kali/dem2o.pcap, link-type EN10MB (Ethernet), snapshot length 262144
13:57:34.068436 IP 94.237.50.147.443 > 192.168.238.128:38980: Flags [P.], seq 5830509781:583051137, ack 394764201, win 64240, length 159
0x0000: 4500 00c7 f6b5 0000 8006 02d2 5eed 3293 E.....^..2.
0x0010: c0a8 ee80 01bb 9844 22c0 a6e2 1787 9fa9 .....D.....
0x0020: 5010 faf0 c412 0000 1703 0300 9af5 8f1c P.....
0x0030: 72c8 4876 9a14 6371 54a5 c28c f4f4 56d5 r.Hv..cqT....V.
0x0040: 91c7 5f7e cf18 b48a eb73 80a1 73cf 7885 .._.....S..S.X.
0x0050: 7620 5307 d0d4 47c2 965b 7d2c 077e 8860 v.S...G..[.]...
0x0060: a154 0302 5951 65e7 412f 3299 b5f3 8467 .T...Yqe-A/2...g
0x0070: bc98 d1ba e7a4 8a63 7b0b 6645 d783 6e7a .....c{.fe..nz
0x0080: 270f 6620 c4a2 5006 434b bdd5 7365 ac0e 'f(...^..CK...se..
0x0090: ae43 9ebf a69c ed11 0eb7 8750 e73f 7ada .C.....P.?z.
0x00a0: 4728 886e a2a3 6848 6bf7 8a80 05fd 92cf G(.n...hKk.....
0x00b0: e0a3 e2ab 6890 1255 c389 0d64 86c5 0e75 ...h...U...d...u
```

On the right, a web browser window shows a page titled "Capturing With Tcpdump". The page contains the following text:

Our team members have given us a PCAP they captured while surveying another section of the enterprise, read the PCAP file into tcpdump, and modify our view of the PCAP to help us determine what is happening. We can disable hostname and port resolution for simplicity and ensure we see any TCP sequence and acknowledgment numbers in absolute values. For the sake of the lab, utilize the PCAP file we created in the previous step for this task.

Click to show answer

```
sirkirika@htb[/htb]$ tcpdump -nnSxR [file/to/read.pcap]
```

The switches used above will not resolve hostnames or port numbers, apply for absolute sequence numbers, and show contents in Hex and ASCII when reading from the PCAP file.

When done with the tasks above, please answer the questions at the bottom of the section to test our understanding.

Filtering and Advanced Syntax Options

Host Filter

The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the following commands and output:

```
kali@kali:~$ sudo tcpdump -i eth0 src host 192.168.238.128
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 byte s
14:09:58.672072 IP 192.168.238.128.38980 > 94-237-50-147.uk-lon1.upcloud.ho
st.https: Flags [P.], seq 395441741:395441779, ack 592318010, win 65535, le
ngth 38
14:09:58.673772 IP 192.168.238.128.38980 > 94-237-50-147.uk-lon1.upcloud.ho
st.https: Flags [P.], seq 38776, ack 1, win 65535, length 38
14:09:58.727557 IP 192.168.238.128.51083 > 192.168.238.2.domain: 28087+ PTR
? 147.50.237.94.in-addr.arpa. (44)
14:09:58.740916 IP 192.168.238.128.58291 > 192.168.238.2.domain: 18042+ PTR
? 128.238.168.192.in-addr.arpa. (46)
14:09:58.757769 IP 192.168.238.128.38980 > 94-237-50-147.uk-lon1.upcloud.ho
st.https: Flags [P.], seq 76114, ack 343, win 65535, length 38
14:09:58.759777 IP 192.168.238.128.38980 > 94-237-50-147.uk-lon1.upcloud.ho
st.https: Flags [P.], seq 1147152, ack 343, win 65535, length 38
14:09:58.825638 IP 192.168.238.128.42544 > 192.168.238.2.domain: 59482+ PTR
? 2.238.168.192.in-addr.arpa. (44)
14:09:58.836323 IP 192.168.238.128.38980 > 94-237-50-147.uk-lon1.upcloud.ho
st.https: Flags [P.], seq 152190, ack 465, win 65535, length 38
14:09:58.836685 IP 192.168.238.128.38980 > 94-237-50-147.uk-lon1.upcloud.ho
st.https: Flags [P.], seq 190228, ack 465, win 65535, length 38
```

On the right, a web browser window shows a page titled "Tcpdump Packet Filtering". The page contains the following text:

and / && together. for example, src host AND port.

or or allows for a match on either of two conditions. It does not have to meet both. It can be tricky.

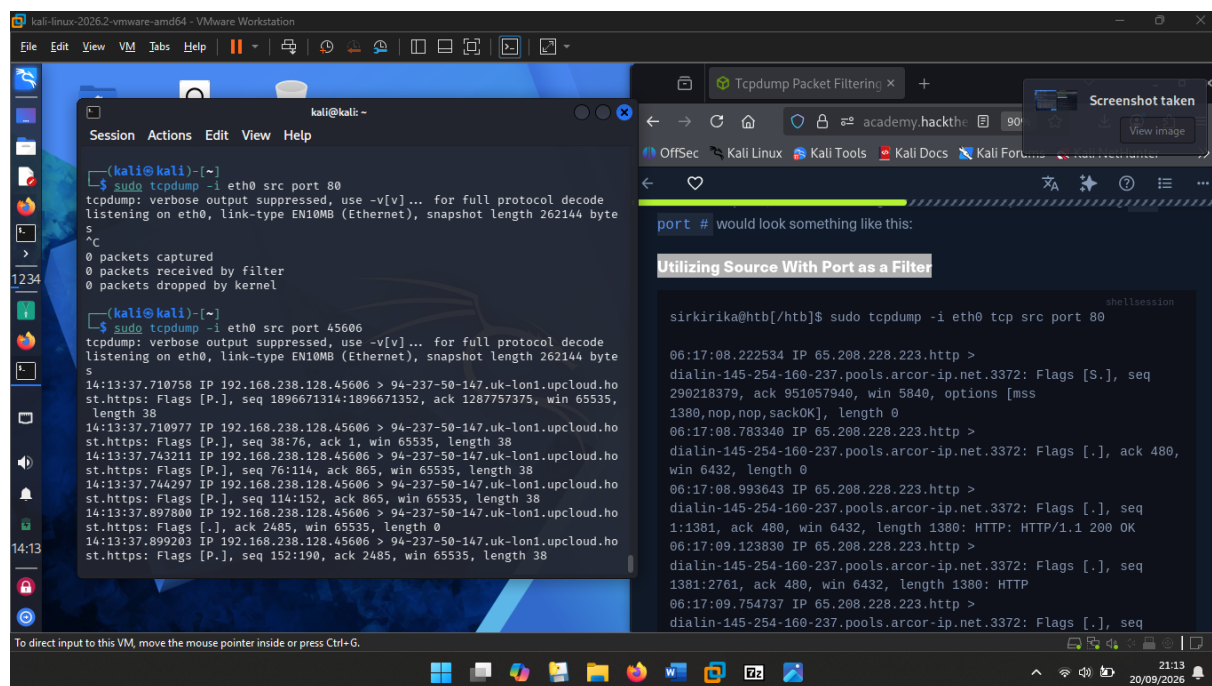
not not is a modifier saying anything but x. For example, not UDP.

With these filters, we can filter the network traffic on most properties to facilitate the analysis. Let us look at some examples of these filters and how they look when we use them. When using the **host** filter, whatever IP we input will be checked for in the source or destination IP field. This can be seen in the output below.

Host Filter

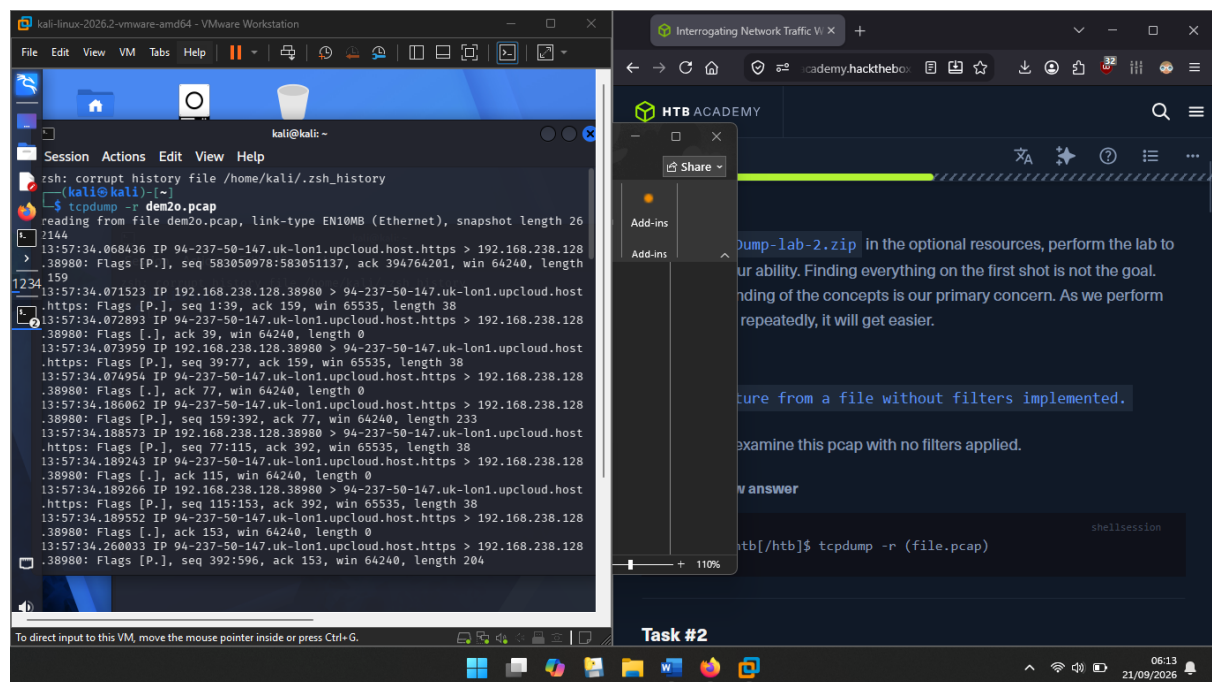
```
sirkirika@htb[/htb]$ ### Syntax: host [IP]
sirkirika@htb[/htb]$ sudo tcpdump -i eth0 host 172.16.146.2
```

Utilizing Source With Port as a Filter



Tasks

Read a capture from a file without filters implemented.



What type of traffic do we see?

Common protocols:

The unfiltered capture showed a mix of TCP, UDP, DNS, and HTTP traffic, with TCP making up the majority of packets.

Filter out traffic.

The screenshot shows a Kali Linux virtual machine (VM) running in VMware Workstation. The terminal window displays a list of network traffic captured by tcpdump, showing various protocols like HTTP, HTTPS, and DNS. The user has entered the command `sudo tcpdump -r dem20.pcap udp and port 53` to filter the traffic. The web browser window shows a page titled "Interrogating Network Traffic" with a search bar and a list of questions related to the traffic analysis. The questions include: "It's time to clear some of this data out now. Reload the pcap file and filter out all traffic that is not DNS. What can you see?", "Who is the DNS server for this segment?", "What domain name/s were requested in the pcap file?", and "What type of DNS Records could be seen?". The page also includes a "Click to show answer" button and a "Keep in mind" section about DNS protocols.

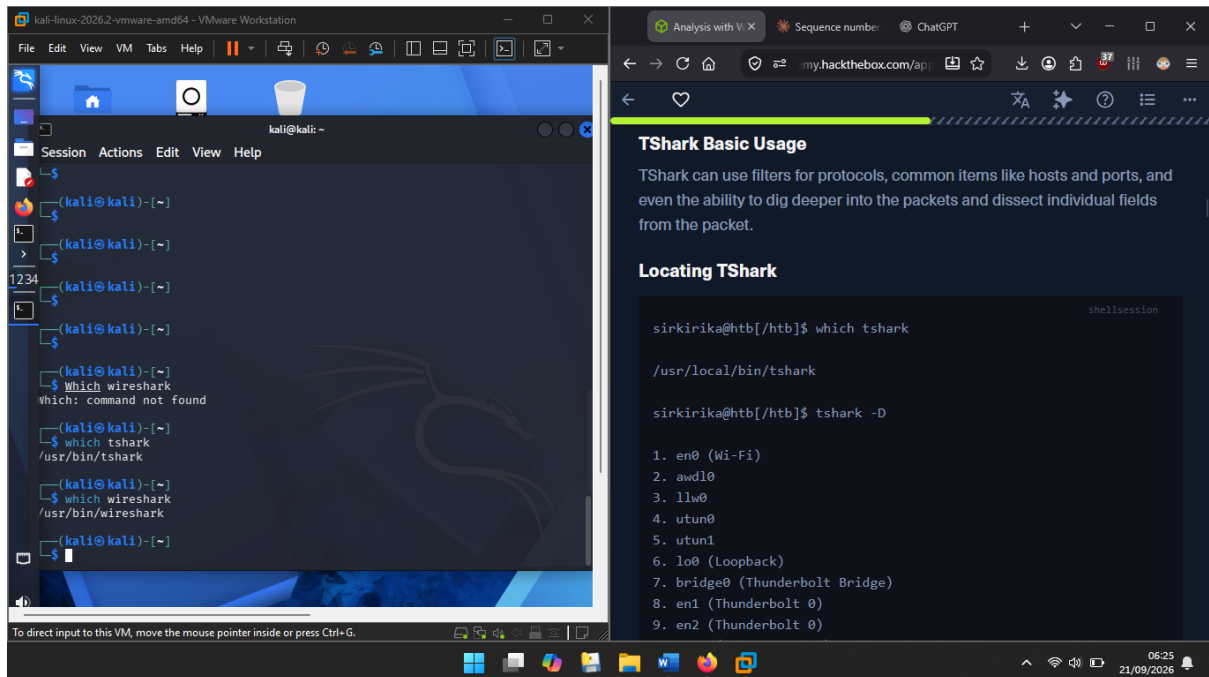
Summary

Through this lab, we expanded our horizons while utilizing TCPDump to analyze PCAP traffic. We learned how to capture and display filters effectively, dissected traffic to determine what protocols were running in the environment, and even gleaned some critical information about our enterprise segments, DNS, and Webservers.

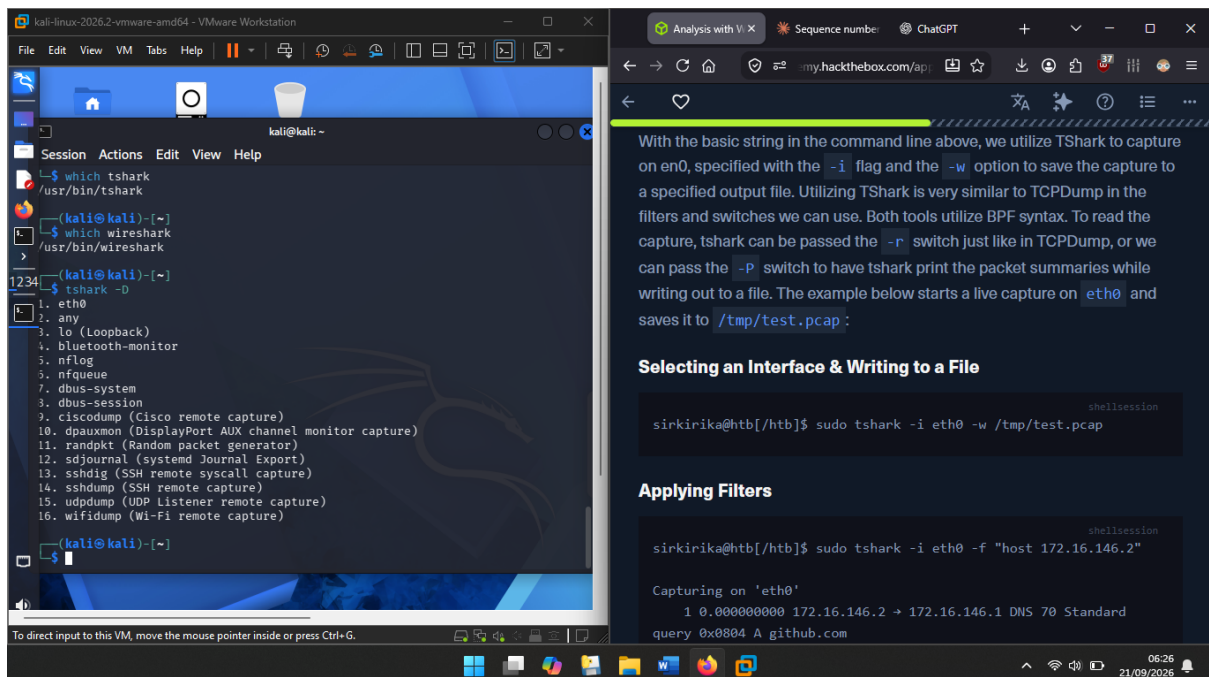
Part 2: Tshark commands/swirches(Analysis with Wireshark)

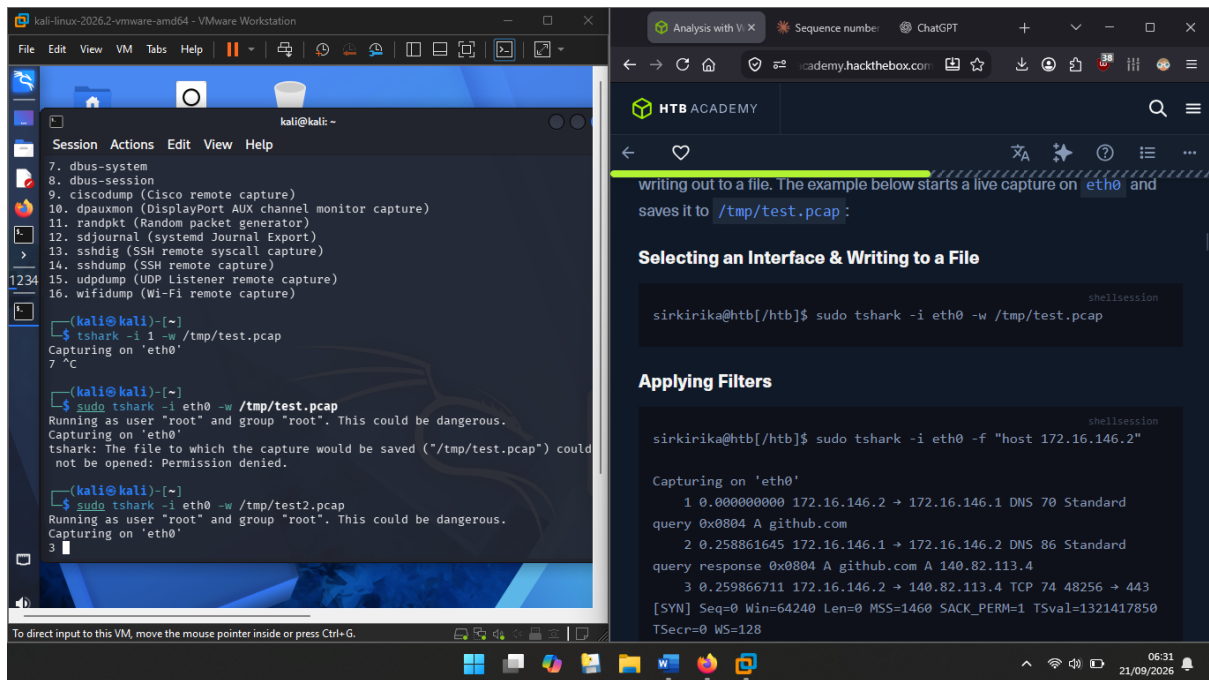
Locating Wireshark

Locating Wireshark from the applications menu on the host.

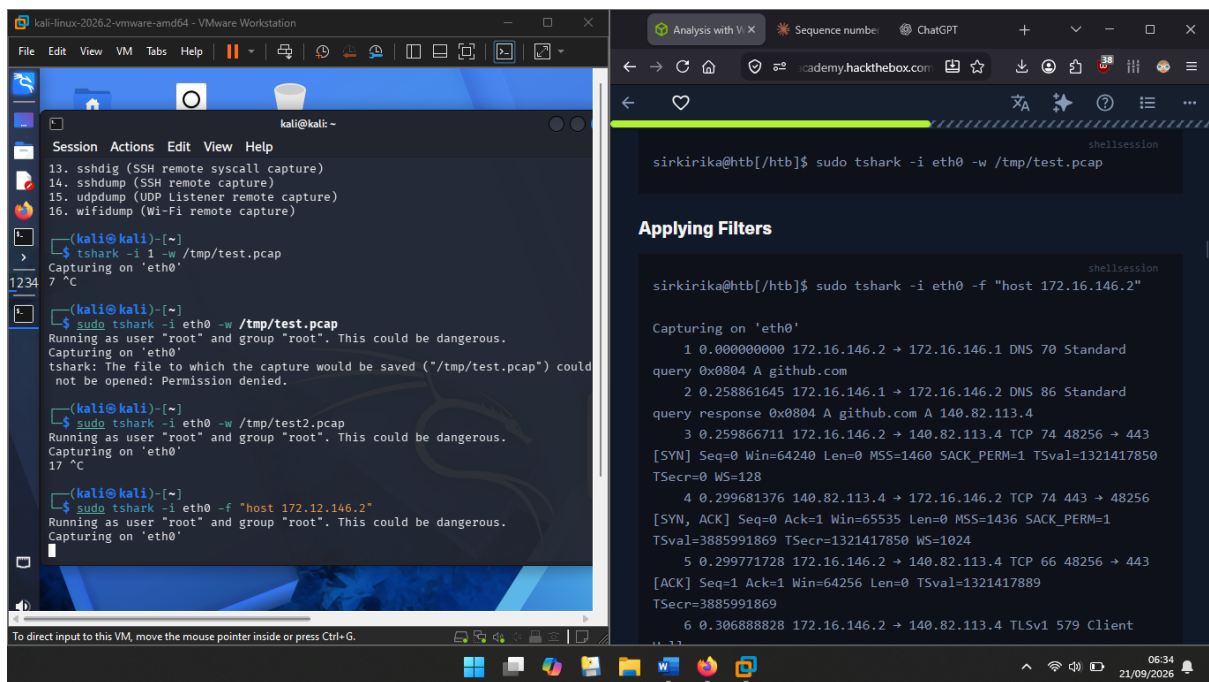


Locating TShark and Selecting an Interface & Writing to a File

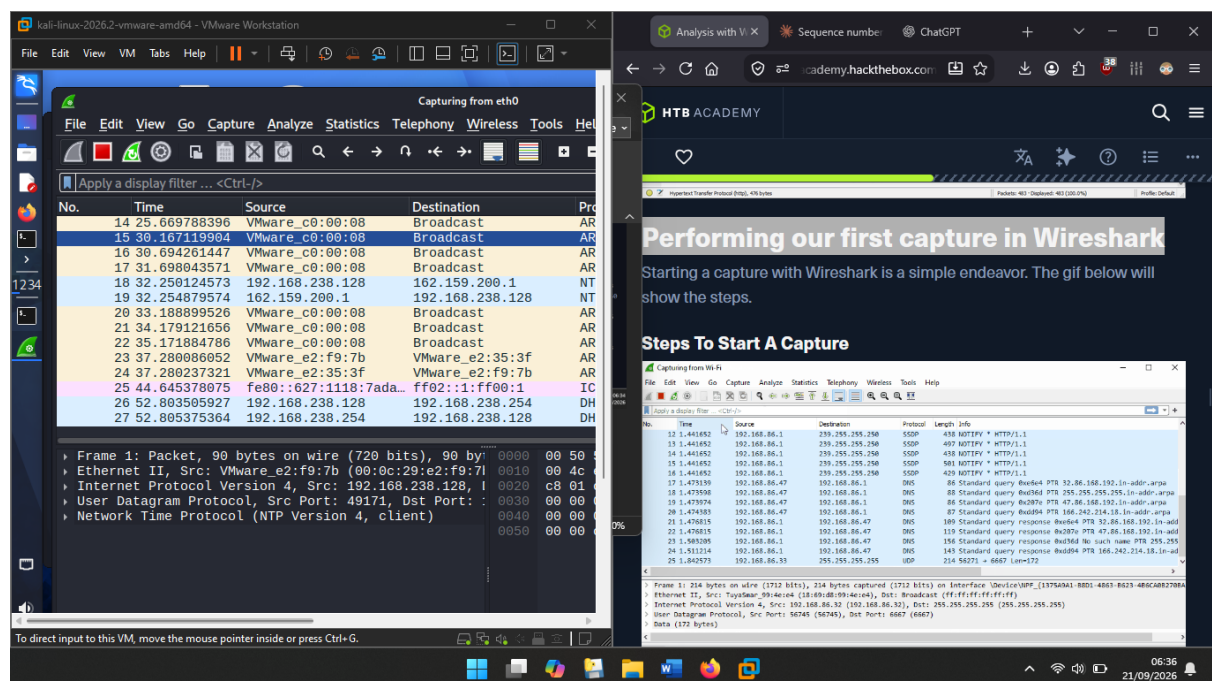




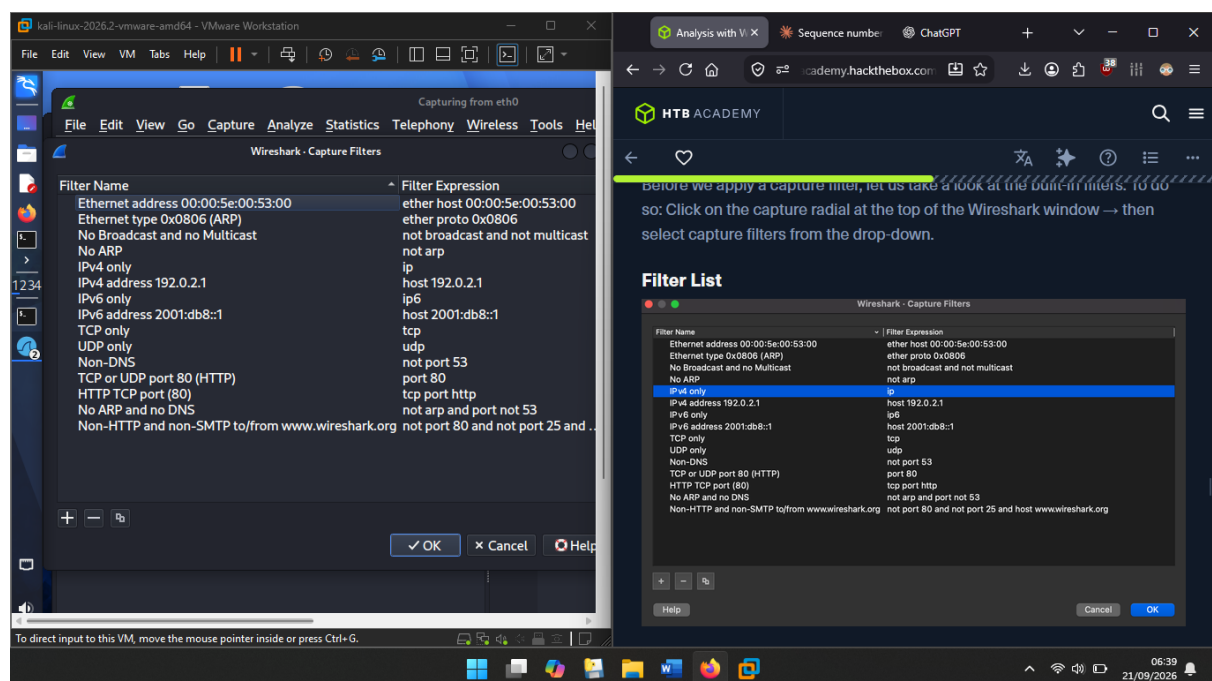
Applying Filters



Performing our first capture in Wireshark



Filter List

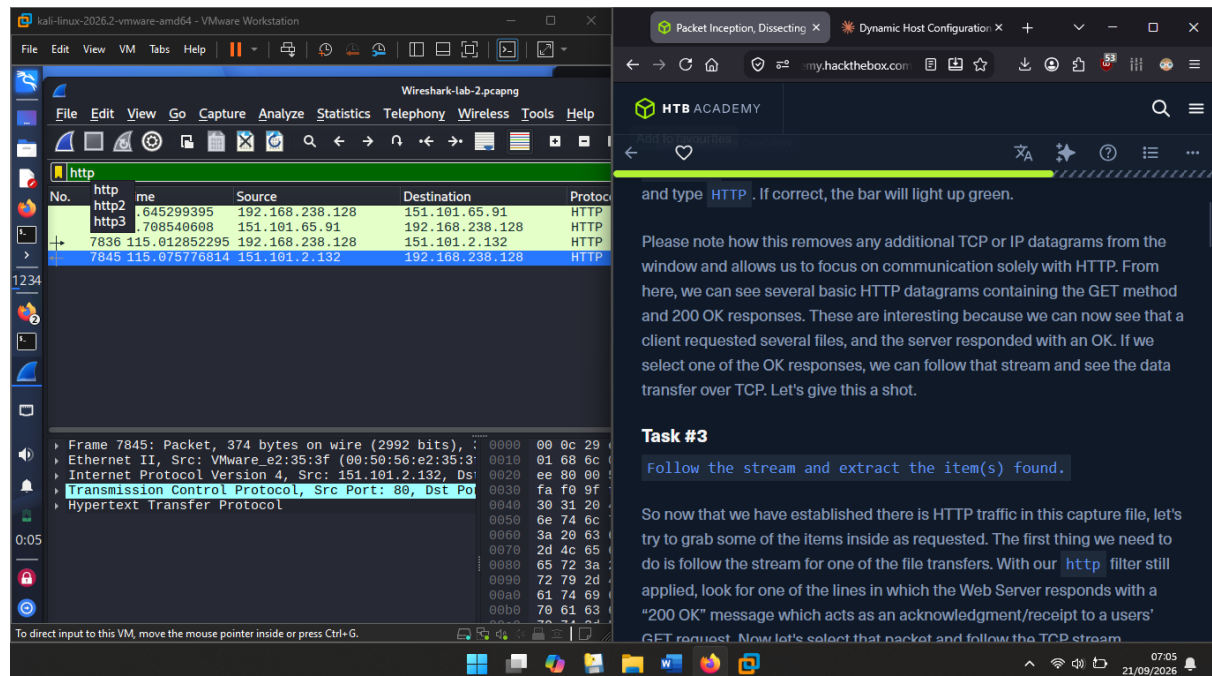


Summary

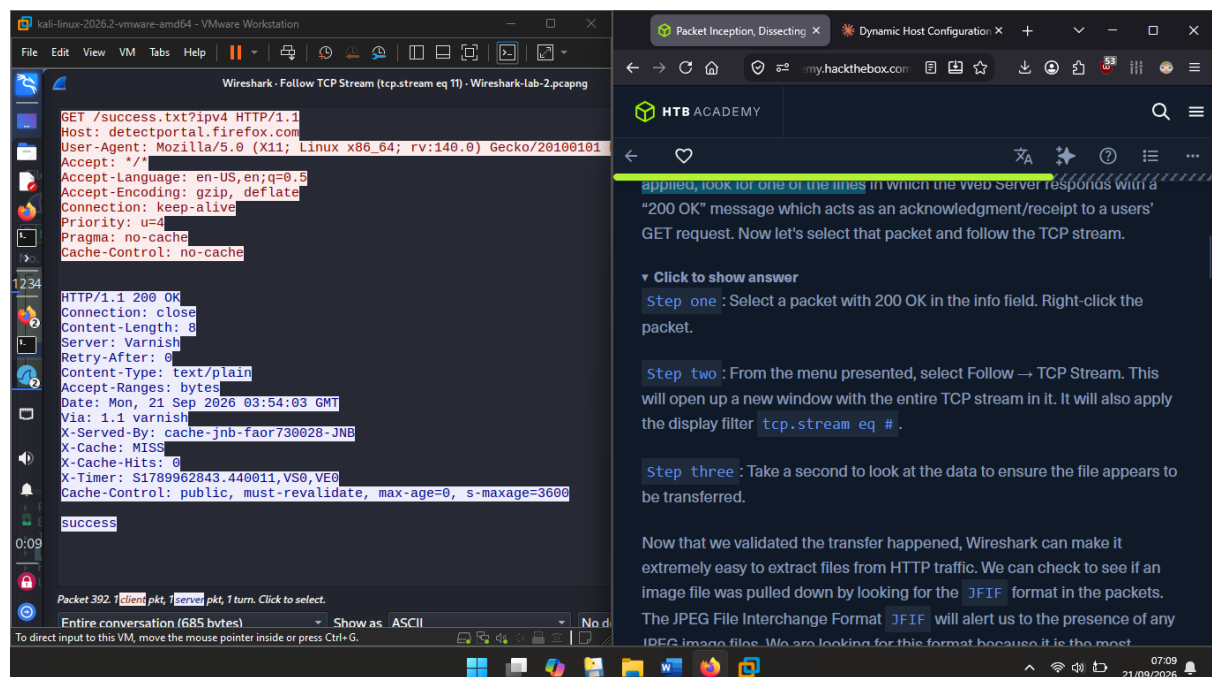
We have familiarized ourselves with the command bar and action bar and what resides within each respective tab. One should understand how to select an interface to capture with and successfully start a capture with a filter applied.

Part 3: Test and apply Gui wireshark filters.

Apply http filter



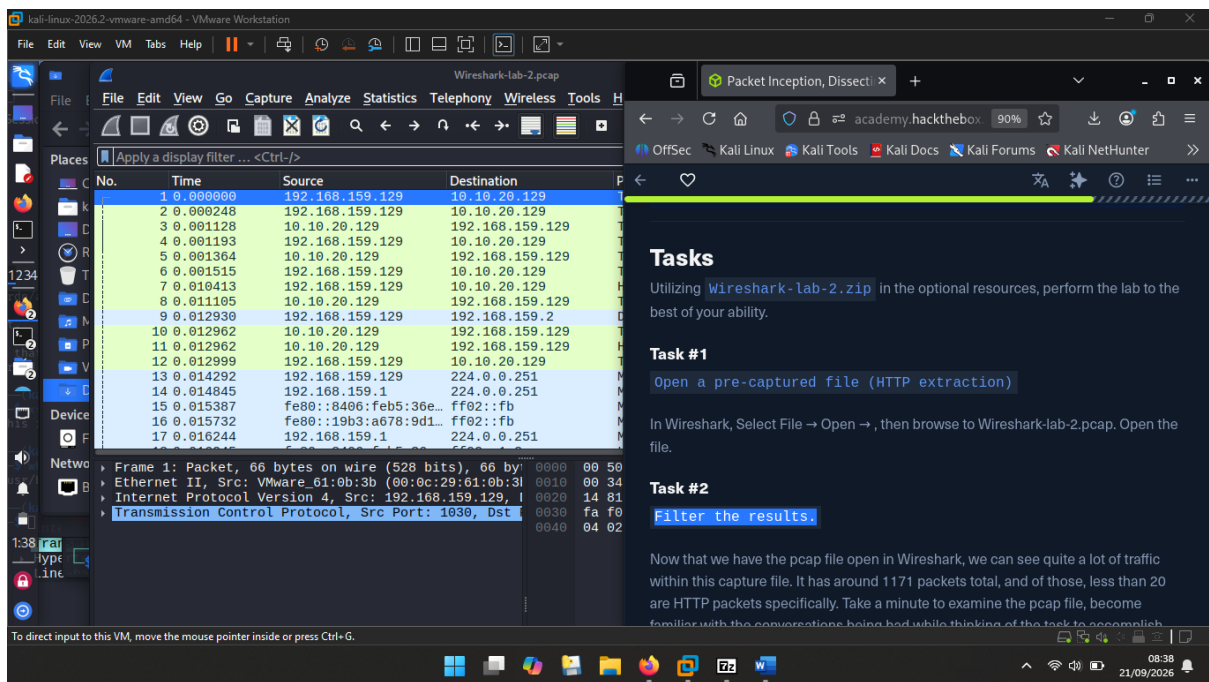
TCP Stream.



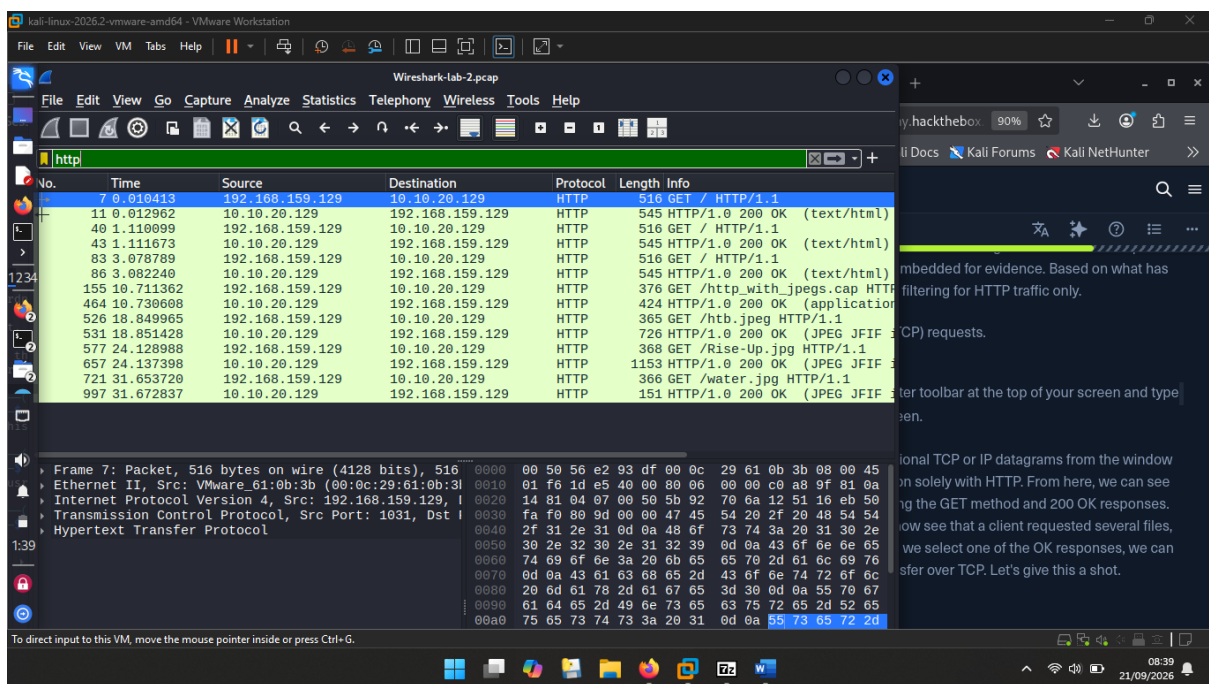
Packet Inception, Dissecting Network Traffic With Wireshark

Utilizing Wireshark-lab-2.zip

Open a pre-captured file (HTTP extraction)

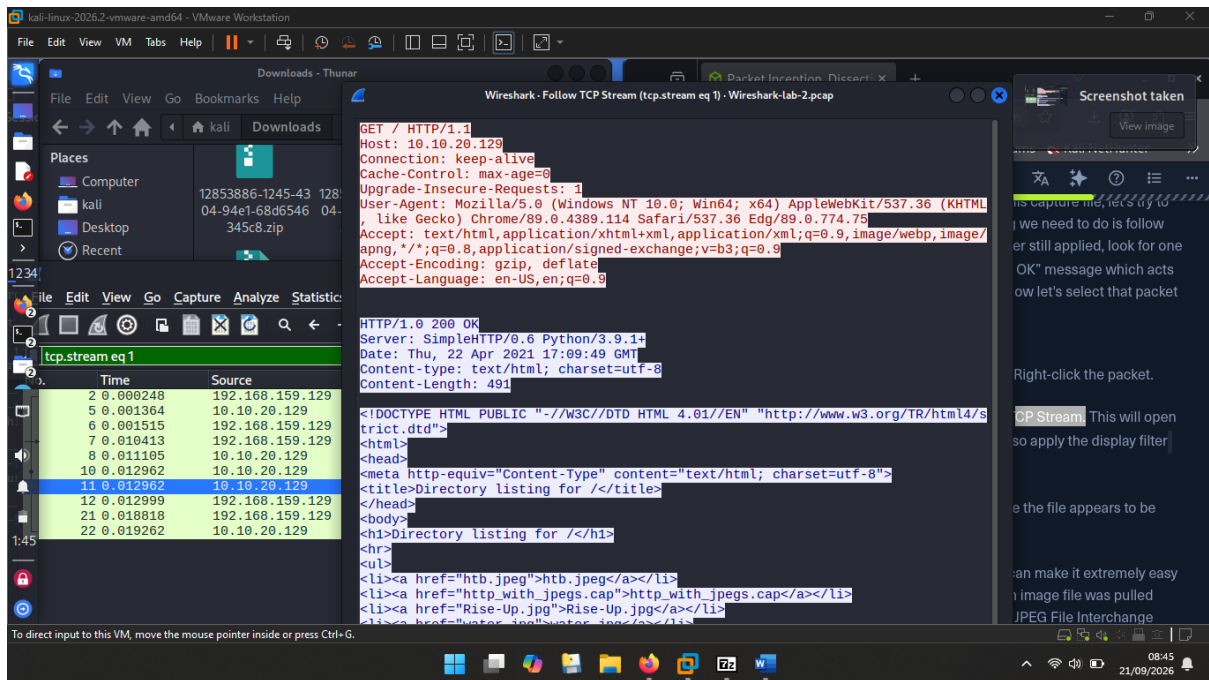


Filter the results.

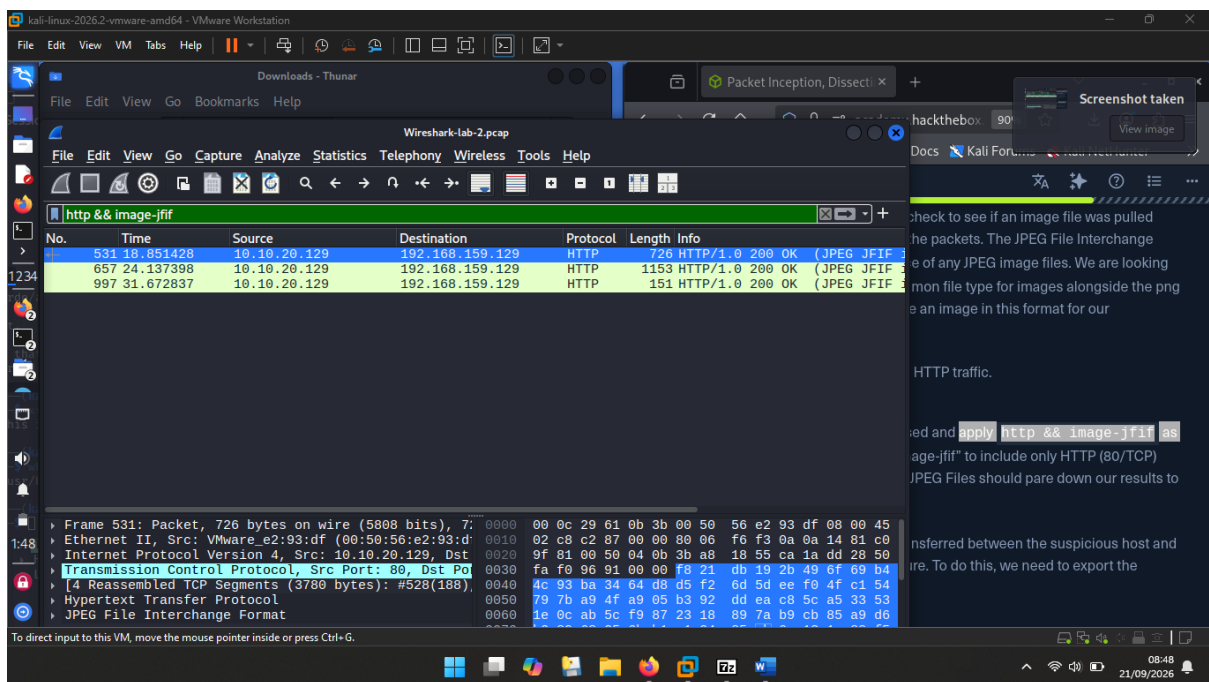


Follow the stream and extract the item(s) found.

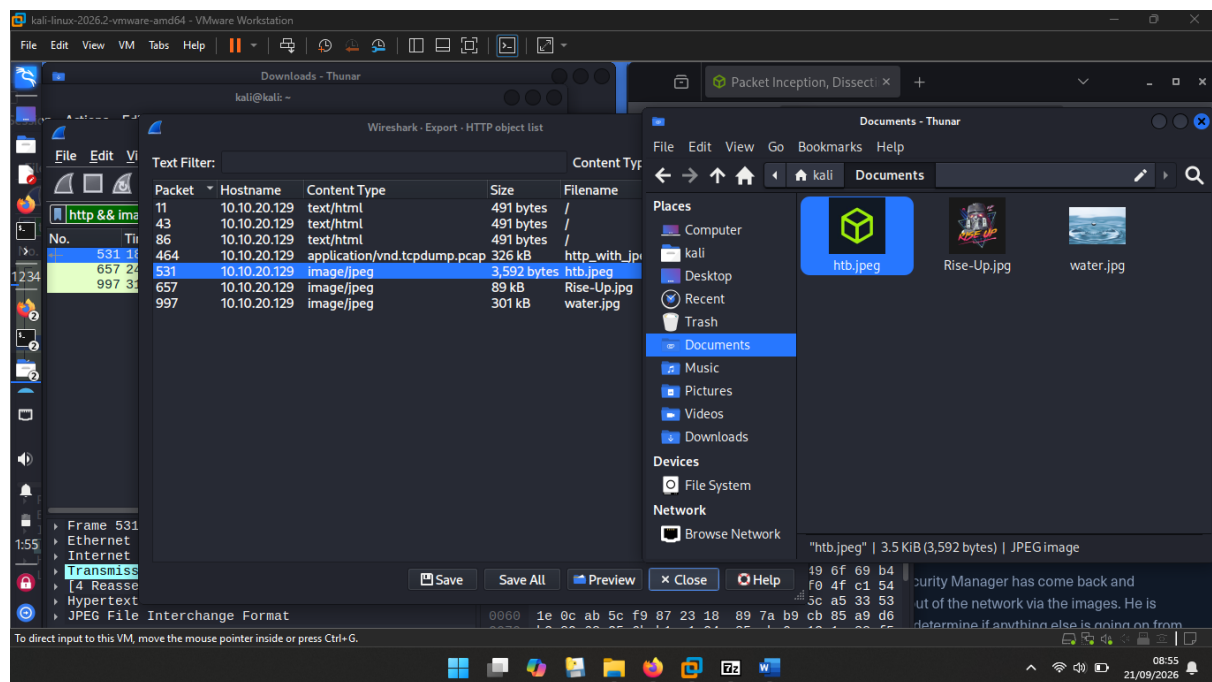
TCP Stream.



apply http && image-jiff as a display filter.

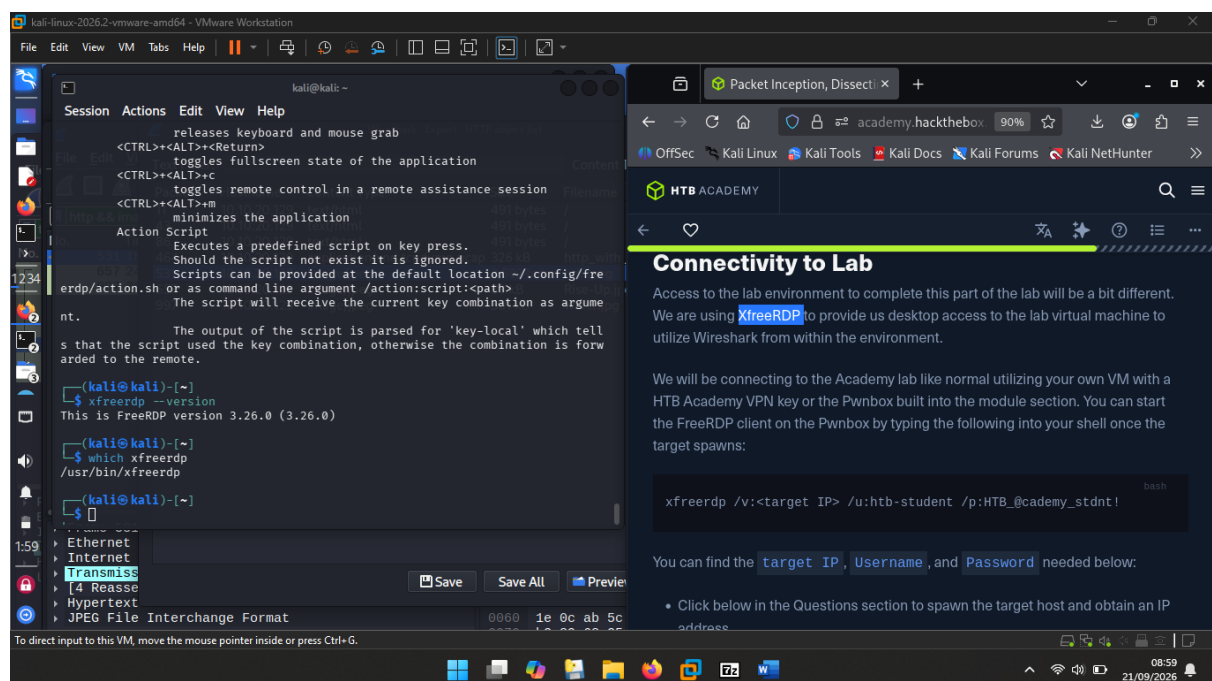


Check for the presence of JFIF files in the HTTP traffic.

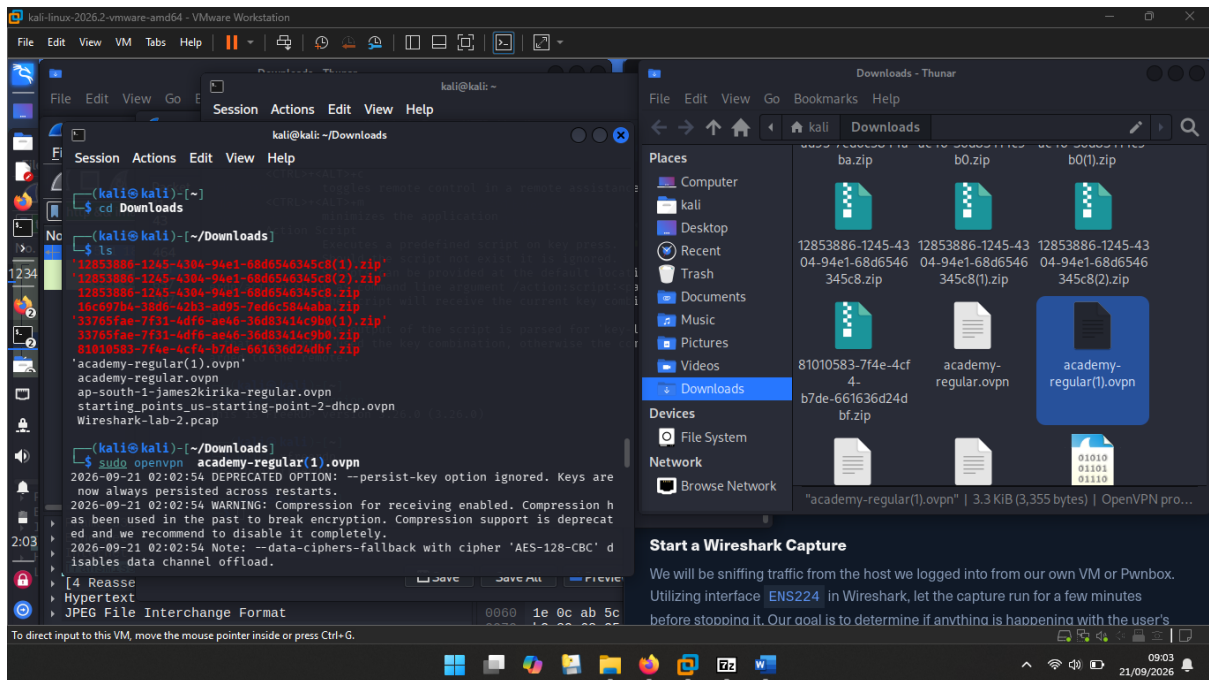


Live Capture and Analysis Connectivity to Lab

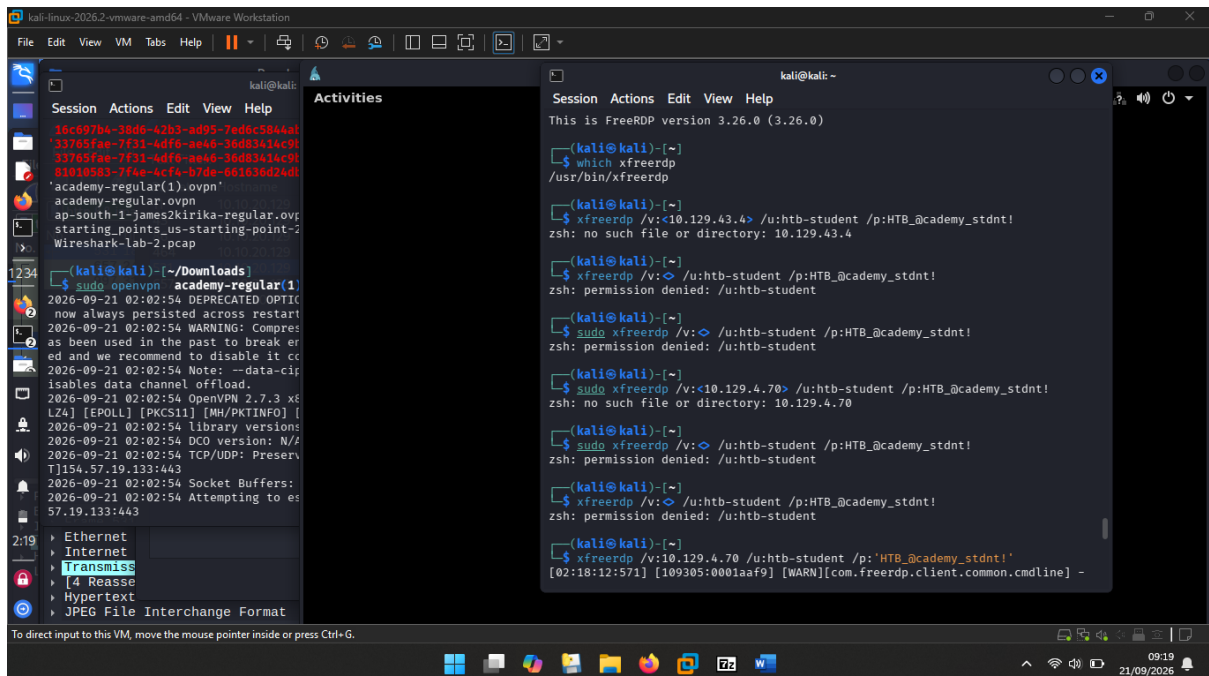
Check XfreerDP



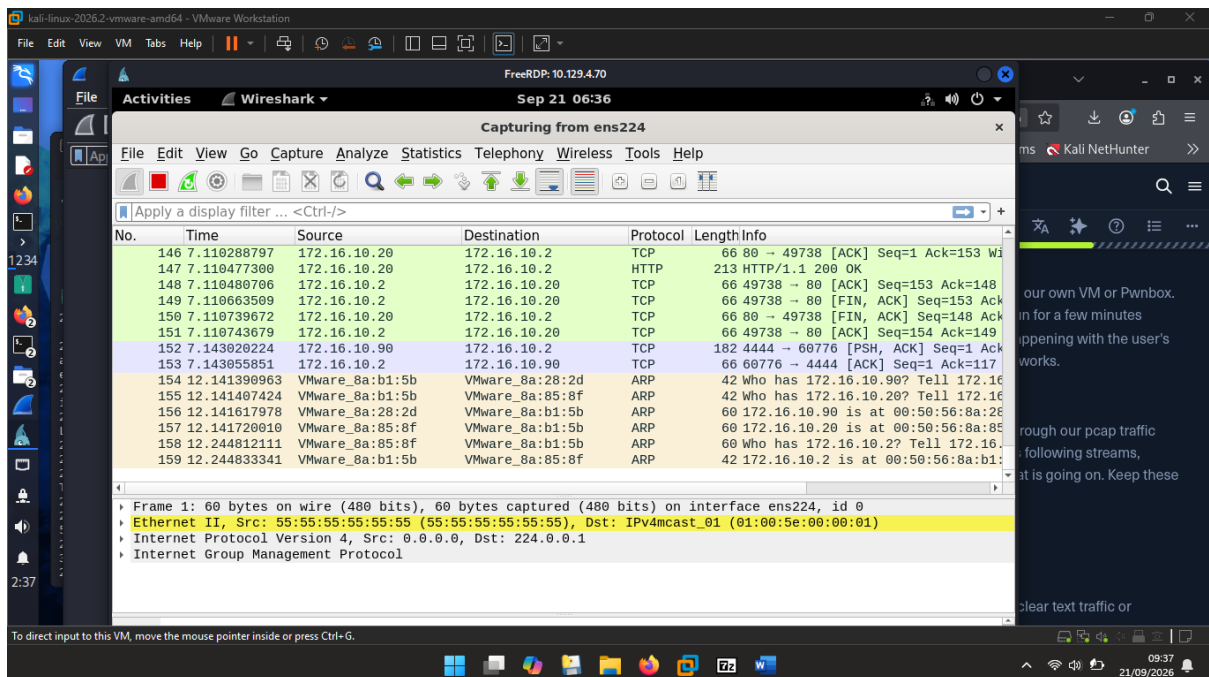
Running vpn



Connectivity to Lab

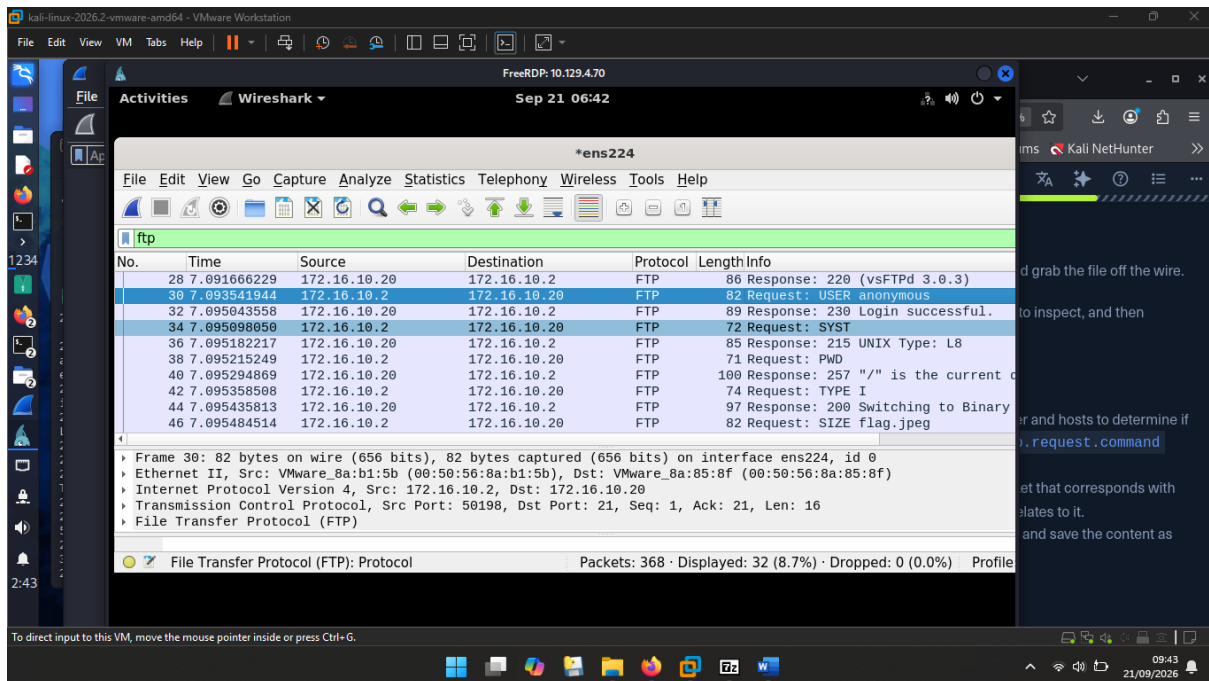


Running wireshark in rdp



Self Analysis

ftp filter



FTP Traffic Analysis Summary

The FTP server was 172.16.10.20:21 (vsFTPD 3.0.3); the client was 172.16.10.2:50198. Login was anonymous (USER anonymous, 230 Login successful), and the entire session, including

credentials, commands, and file contents, was transmitted in cleartext. The file flag.jpeg (180,556 bytes) was pulled via RETR in passive mode.

Ftp-Data filter

Wireshark packet capture showing FTP data traffic on interface ens224. The packet list shows multiple FTP data packets (PASV) from 172.16.10.20 to 172.16.10.2. The packet details pane shows the structure of an FTP data packet, including Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol (Seq: 40878, Dst Port: 52258). The packet bytes pane shows the raw data.

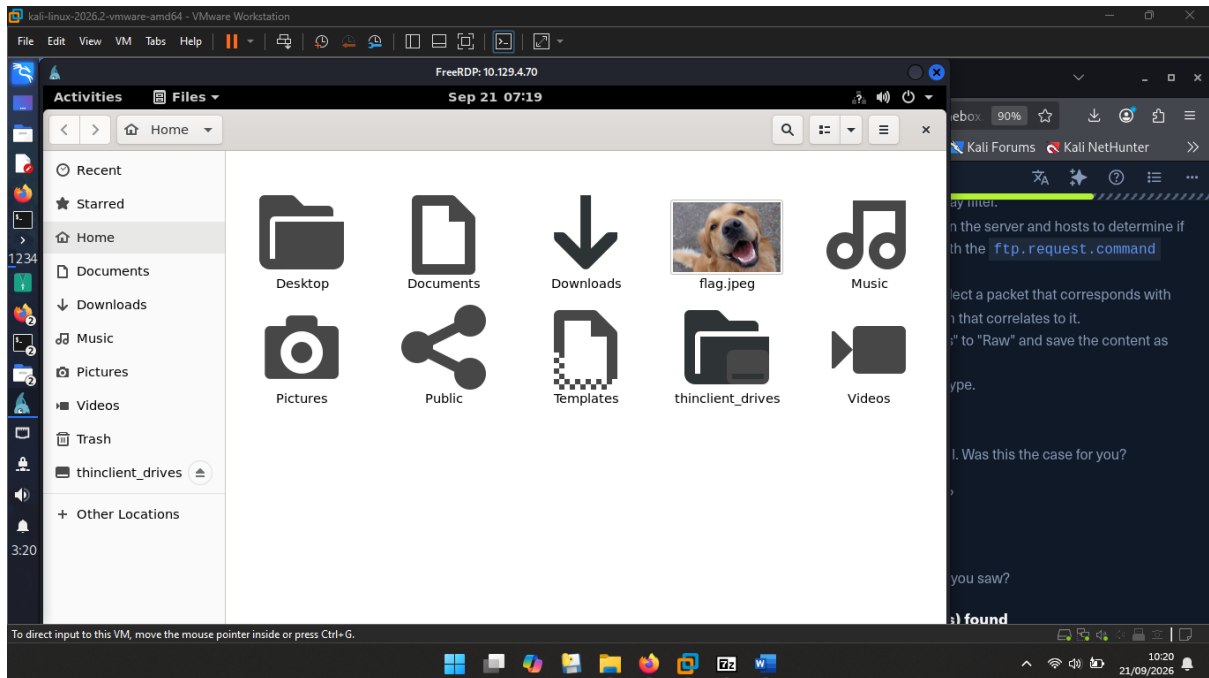
No.	Time	Source	Destination	Protocol	Length	Info
65	7.096417542	172.16.10.20	172.16.10.2	FTP-DA...	7306	FTP Data: 7240 bytes (PASV)
67	7.096437089	172.16.10.20	172.16.10.2	FTP-DA...	7306	FTP Data: 7240 bytes (PASV)
69	7.096479168	172.16.10.20	172.16.10.2	FTP-DA...	10202	FTP Data: 10136 bytes (PASV)
71	7.096492793	172.16.10.20	172.16.10.2	FTP-DA...	14546	FTP Data: 14480 bytes (PASV)
73	7.096506239	172.16.10.20	172.16.10.2	FTP-DA...	4410	FTP Data: 4344 bytes (PASV)
75	7.096547346	172.16.10.20	172.16.10.2	FTP-DA...	2962	FTP Data: 2896 bytes (PASV)
77	7.096563326	172.16.10.20	172.16.10.2	FTP-DA...	17442	FTP Data: 17376 bytes (PASV)
79	7.096578304	172.16.10.20	172.16.10.2	FTP-DA...	14546	FTP Data: 14480 bytes (PASV)
80	7.096587070	172.16.10.20	172.16.10.2	FTP-DA...	5762	FTP Data: 5696 bytes (PASV)
82	7.096604873	172.16.10.20	172.16.10.2	FTP-DA...	11650	FTP Data: 11584 bytes (PASV)
84	7.096707416	172.16.10.20	172.16.10.2	FTP-DA...	20338	FTP Data: 20272 bytes (PASV)
86	7.096728034	172.16.10.20	172.16.10.2	FTP-DA...	31922	FTP Data: 31856 bytes (PASV)
87	7.096740007	172.16.10.20	172.16.10.2	FTP-DA...	31922	FTP Data: 31856 bytes (PASV)
88	7.096748122	172.16.10.20	172.16.10.2	FTP-DA...	1166	FTP Data: 1100 bytes (PASV)
280	187.160297145	172.16.10.20	172.16.10.2	FTP-DA...	7306	FTP Data: 7240 bytes (PASV)

ftp-data Tcp Stream

Wireshark packet capture showing FTP data traffic on interface ens224. The packet list shows multiple FTP data packets (PASV) from 172.16.10.20 to 172.16.10.2. The packet details pane shows the structure of an FTP data packet, including Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol (Seq: 40878, Dst Port: 52258). The packet bytes pane shows the raw data.

No.	Time	Source	Destination	Protocol	Length	Info
56	7.0960433	172.16.10.20	172.16.10.2	FTP-DA...	7306	FTP Data: 7240 bytes (PASV)
57	7.0961336	172.16.10.20	172.16.10.2	FTP-DA...	7306	FTP Data: 7240 bytes (PASV)
58	7.0961526	172.16.10.20	172.16.10.2	FTP-DA...	10202	FTP Data: 10136 bytes (PASV)
65	7.0964175	172.16.10.20	172.16.10.2	FTP-DA...	14546	FTP Data: 14480 bytes (PASV)
66	7.0964236	172.16.10.20	172.16.10.2	FTP-DA...	4410	FTP Data: 4344 bytes (PASV)
67	7.0964376	172.16.10.20	172.16.10.2	FTP-DA...	2962	FTP Data: 2896 bytes (PASV)
68	7.0964409	172.16.10.20	172.16.10.2	FTP-DA...	17442	FTP Data: 17376 bytes (PASV)
69	7.0964791	172.16.10.20	172.16.10.2	FTP-DA...	14546	FTP Data: 14480 bytes (PASV)
70	7.0964836	172.16.10.20	172.16.10.2	FTP-DA...	5762	FTP Data: 5696 bytes (PASV)
71	7.0964927	172.16.10.20	172.16.10.2	FTP-DA...	11650	FTP Data: 11584 bytes (PASV)
72	7.0964977	172.16.10.20	172.16.10.2	FTP-DA...	20338	FTP Data: 20272 bytes (PASV)
73	7.0965062	172.16.10.20	172.16.10.2	FTP-DA...	31922	FTP Data: 31856 bytes (PASV)
74	7.0965091	172.16.10.20	172.16.10.2	FTP-DA...	31922	FTP Data: 31856 bytes (PASV)
75	7.0965473	172.16.10.20	172.16.10.2	FTP-DA...	1166	FTP Data: 1100 bytes (PASV)
76	7.0965506	172.16.10.20	172.16.10.2	FTP-DA...	7306	FTP Data: 7240 bytes (PASV)

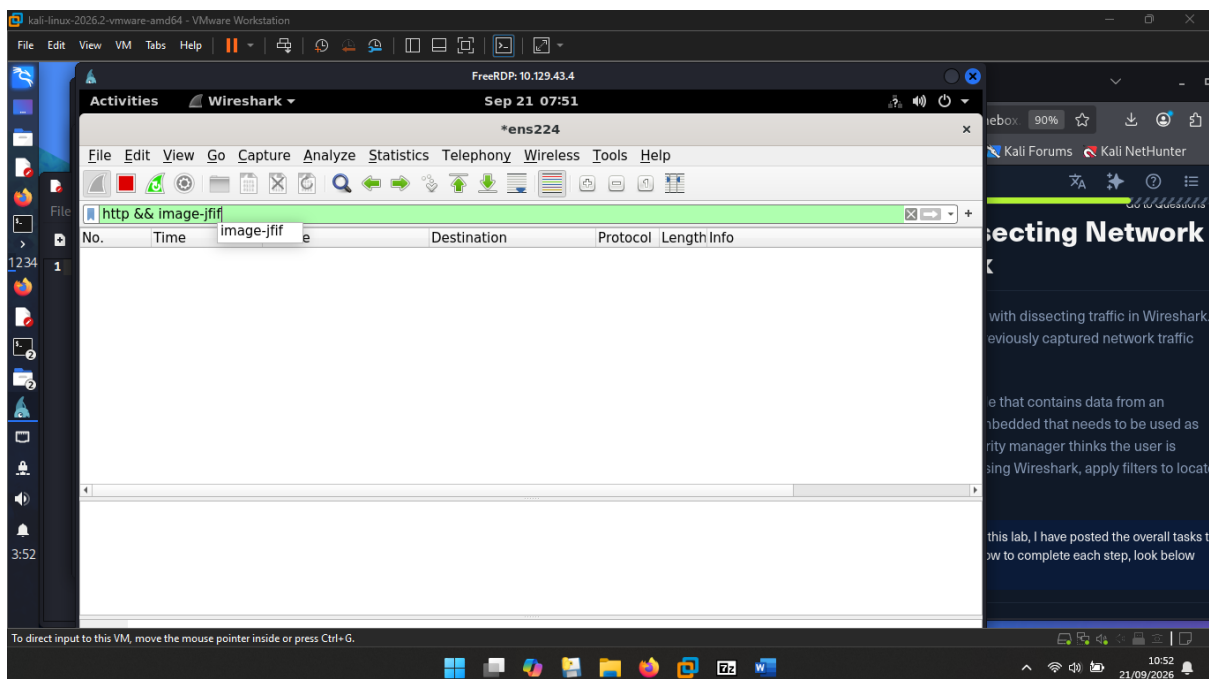
saving raw data



the file is jpeg format

HTTP Analysis

Apply the filter "http && image-jfif"

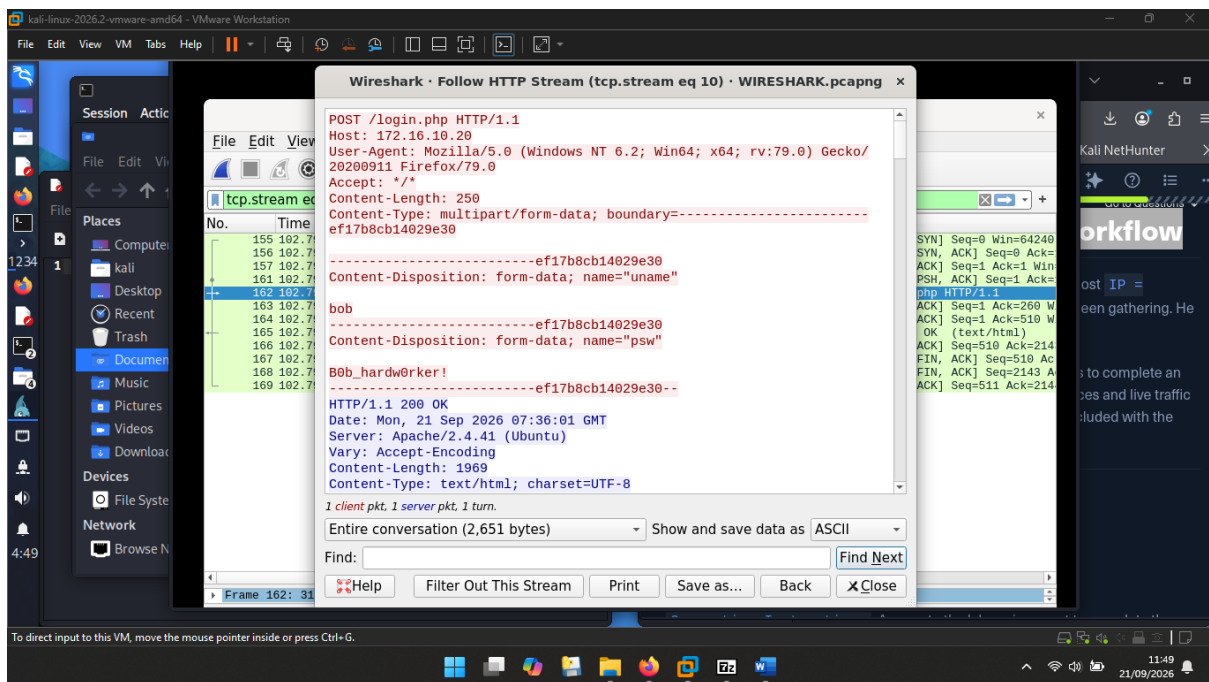


Empty results

No results were returned for http & image-jfif, confirming the image was not transferred over HTTP in this capture. It was retrieved via FTP instead, as shown in the earlier FTP-data extraction.

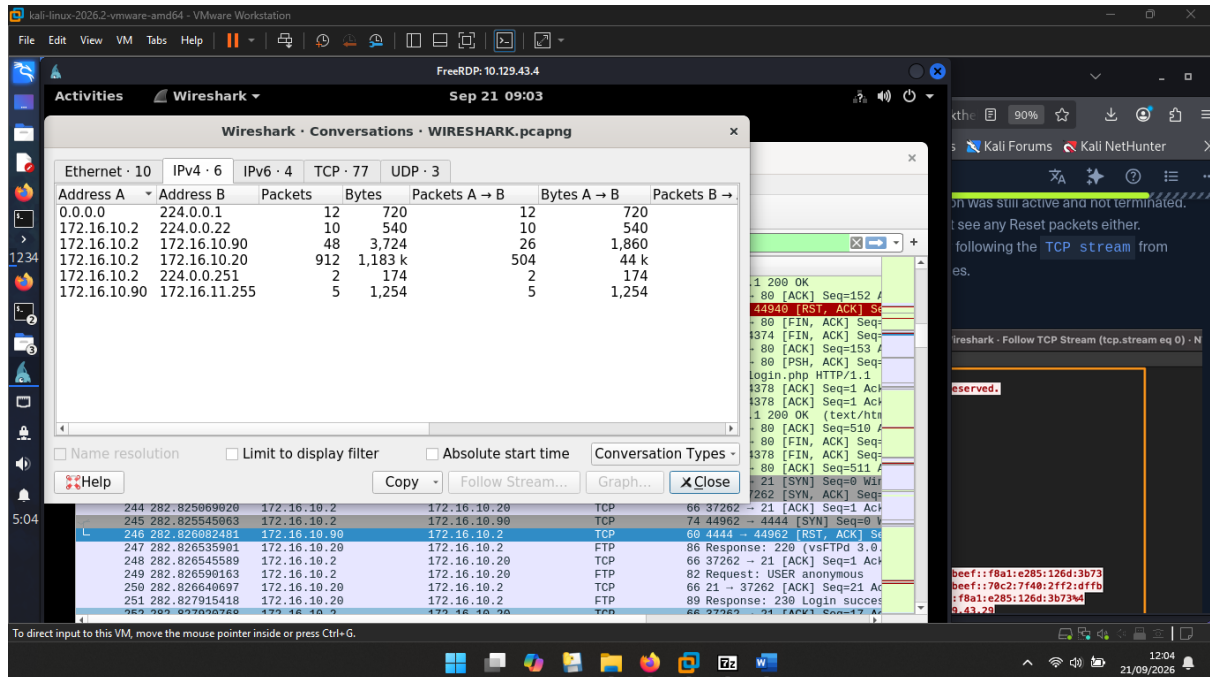
Investigate which employee is the suspect.

Apply filter "http" and right click on "Post" then follow the HTTP stream. This lands you on the screen shown below, where the suspect's username is visible in plain text; in this case, bob.



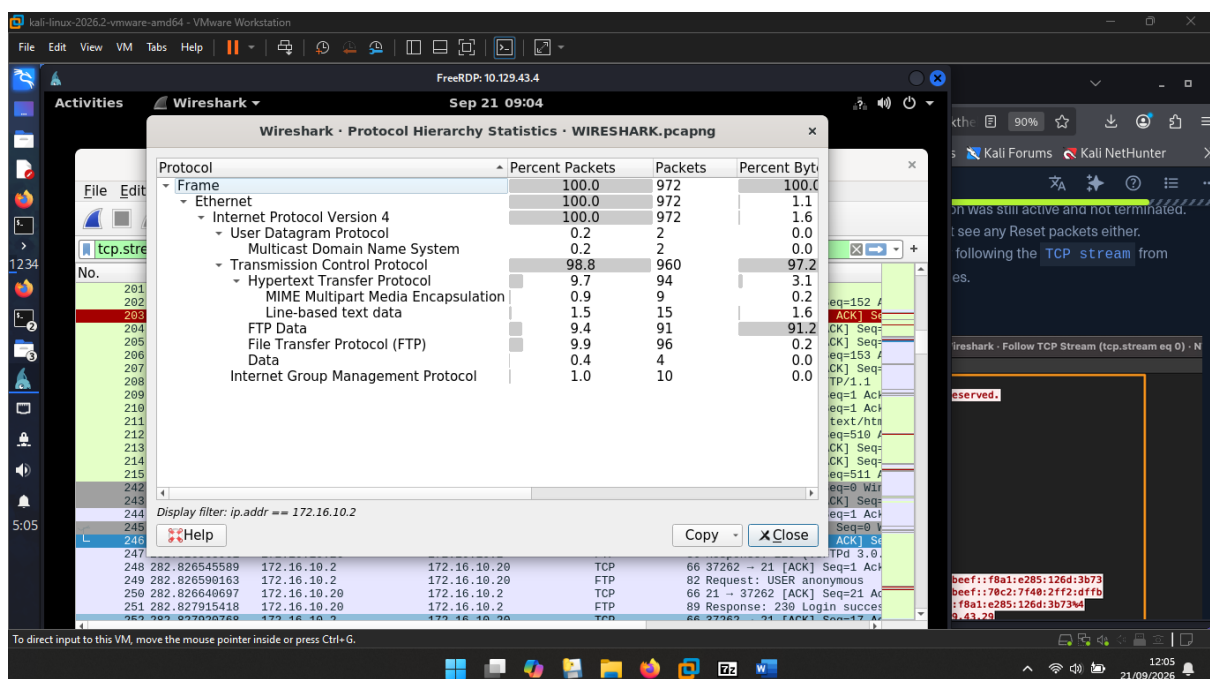
Further investigation

View conversations



we can see there are only three conversations captured in this pcap file, and they all pertain to our suspicious host, 172.16.10.2.

View protocol hierarchy



We can see here that this PCAP is only TCP traffic.

Summary

we were able to open previously captured .pcap files, apply display filters, follow streams, and extract items from the capture file. We captured new traffic on our own and applied filters to isolate specific traffic types, which reinforced our understanding of the tools and confirmed our ability to answer the questions using the traffic we captured

Conclusion

Working through tcpdump and Wireshark side by side made clear how much of network security still comes down to visibility rather than complexity. Filtering down a noisy capture to just the traffic that matters, whether by protocol, host, or port, turned an overwhelming packet list into something actually readable. The FTP exercise showed how anonymous, unauthenticated file transfers can move data off a network without leaving much of a trail, and the HTTP analysis was a clear reminder that plaintext login forms still expose real credentials in practice, not just in theory. Correlating the suspicious host across FTP, HTTP, and the conversation and protocol hierarchy views tied the individual findings into a single, coherent picture of what happened on that host. This module built practical comfort with Wireshark's filtering and stream-following workflow, and a sharper instinct for what to check first when starting from an unfamiliar capture.

Links:

[Hackthebox](#)

[Linkedin](#)